UNIDAD 4
TAREA 1

Objetivo

En esta actividad, aplicarás los principios de **Programación Orientada a Objetos (POO)** en Python para resolver problemas de manera estructurada, utilizando **clases, herencia, polimorfismo, encapsulación y métodos especiales**.

Ejercicio 1: Creando una Clase Básica (0,5 ptos)

Crea una clase `Coche` con los siguientes atributos y métodos:

- **Atributos**: `marca`, `modelo`, `anio`.
- **Métodos**:
  - `arrancar()`: Imprime un mensaje indicando que el coche ha arrancado.

    Ej. “El coche Mazda CX-5 ha arrancado”

  - `detener()`: Imprime un mensaje indicando que el coche se ha detenido.

    Ej. “El coche Mazda CX-5 se ha detenido”

Ejercicio 2: Encapsulación y Métodos Getters/Setters (1 pto)

Extiende la clase `Coche` para incluir un atributo privado `_kilometraje`.

- **Añade los siguientes métodos**:
  - `get_kilometraje()`: Retorna el kilometraje.

    Ej. “Kilometraje actual 100 Km”

- `set_kilometraje(nuevo_km)`: Permite actualizar el kilometraje solo si el nuevo valor es mayor al actual, es decir, no debe permitir reducir el kilometraje

  Ej. “Kilometraje actualizado a 200 Km.”

Ejercicio 3: Herencia y Sobrescritura de Métodos (2 ptos)

Crea una clase `Electrico` que **herede** de `Coche` e incluya:

- **Atributo adicional**: `bateria` (capacidad en kWh).

  Ej. “La capacidad de la batería es de 100kWh”

- **Sobreescribe el método `arrancar()`** para indicar que el coche eléctrico ha arrancado en modo silencioso.

  Ej. “El coche eléctrico Tesla Model S ha arrancado en modo silencioso”

Ejercicio 4: Polimorfismo y Métodos Especiales (`__str__`) (2 ptos)

Modifica `Coche` y `Electrico` para que tengan un método `__str__()` que devuelva una cadena con la información del coche.

Ej. “Coche: Mazda CX-5 2025 – Kilometraje: 1000 km”

“Coche Eléctrico: Tesla Model S 2024 – Kilometraje: 900 km – Batería: 100kWh”

Ejercicio 5: Gestión de Flota de Vehículos (2 ptos)

Crea una clase `Flota` que gestione una lista de vehículos. Debe incluir:

- Un atributo `vehiculos` que almacene los coches.
- Métodos:
  - `agregar_coche(coche)`: Añade un coche a la flota.
  - `mostrar_flota()`: Muestra todos los coches registrados.

Ejercicio 6 : Registro de Vehículos con Mixin (2,5 ptos)

Añade un sistema de registro de vehículos con fecha de creación y funcionalidad de auditoría.
Crea dos mixins:

- `AuditMixin`: Guarda la fecha de creación del vehículo.

  from datetime import datetime

  mediante un método *mostrar_auditoria* debe permitir ver la fecha de creación del vehículo: “Este vehículo fue registrado el 25-06-2024”

- `LoggingMixin`: Imprime un mensaje cuando se crea un vehículo.

  Ej. “Vehículo Mazda CX-5 registrado con éxito”

Luego, modifica `Coche` y `Electrico` para heredar de estos mixins e implementar la funcionalidad adicional.

Entrega y Evaluación

- **Entrega**: Cada alumno debe enviar un único archivo **pdf** nombrándolo **Apellidos_Nombre_Tarea4_1.pdf,** con los ejercicios resueltos clasificados por apartados..
- **Criterios de evaluación**:
  - Correcta implementación de clases, atributos y métodos.
  - Uso adecuado de encapsulación, herencia y polimorfismo.
  - Código limpio y documentado con comentarios.
  - El desarrollo y la resolución se ajusta a lo visto en la unidad.

- **Calificación**:

  Ejercicio1: 0,5 ptos

  Ejercicio2: 1 pto

  Ejercicio3: 2 ptos

  Ejercicio4: 2 ptos

  Ejercicio5: 2 ptos

  Ejercicio6: 2,5 ptos